5.4 Model Meta properties

You've seen how the `.__str__()` method on a `Model` class changes the corresponding Django Admin listing page. Other properties of the `Model` can also effect the Django Admin. This is another one of those cases where Django takes advantage of a seldom used Python language feature. I heard you like classes—how about a class inside your class?

Django `Model` classes can optionally define a nested class named `Meta`. Django uses the attributes of this class to modify the default behavior of the resulting `Model` object. Some attributes are only used by the Django Admin, and others change the behavior of the `Model` itself, such as modifying the underlying database queries.

The following sections provide details on some of the most popular `Meta` attributes. For a full list, see https://docs.djangoproject.com/en/dev/ref/models/options/.

5.4.1 Sort order

The `ordering` `Meta` attribute specifies the default sort order in response to queries on the object. This impacts both the order of the objects in the Django Admin listing pages as well as any query you run in your code. The underlying SQL gets an `ORDER BY` clause attached automatically. This attribute takes a list of field names and supports a dash (-) prefix to indicate descending sort order, as shown in the following example:

```
class VenueStaff(models.Model):
    # first_name, last_name, employee ID number fields
    ...

    class Meta:
        ordering = ["last_name", "-employee_id"]
```

Sort by last_name and then by employee_id, in reverse order.

The default sort order is none. SQLite appears to return the `Musician` objects in reverse order by ID, but this should not be depended upon. Ordering costs overhead and requires the database to do additional work. It is handy though. To change the default sort order of the `Musician` class to be based on `last_name` and then `first_name`, make the following change in your RiffMates/bands/models.py file:

```
class Musician(models.Model):
    ...

    class Meta:
        ordering = ["last_name", "first_name"]
```

Once you've made the change, try running `Musician.objects.all()` in the REPL or visit the Django Admin `Musician` listing page to see the difference.

5.4.2 Indexes

In chapter 4, you learned about specifying an index in the field declaration of a `Model`. You can also declare indexes via `Meta` attributes. Using this mechanism allows you to have multiple fields participate in a single index.